오픈소스 SW 개발 방법 및 도구
리눅스 한학기 살기

2024105570 박미소
컴퓨터 공학과

1주차 리눅스 설치하기

1주차에서는 보안 학습과 CTF 문제 제작을 위한 실습 환경을 구축하기 위해 VMware Workstation Player 위에 Ubuntu 22.04 LTS (64-bit)를 설치하였다.
Ubuntu는 대표적인 오픈소스 리눅스 배포판으로, 안정적인 LTS(Long Term Support) 제공, 활발한 커뮤니티, 다양한 보안 도구 및 개발 패키지 지원, 그리고 사용자 친화적인 인터페이스 등 실습 환경으로 사용하기 적합한 장점을 갖추고 있다.

특히 나는 모바일 보안과 CTF 문제 제작에 관심이 많기 때문에 리눅스 환경은 필수적이다. 실제 CTF 문제 제작 과정에서는 웹 서버 구성, 파일 권한 관리, 포트 제어, 네트워크 분석, 패킷 캡처, 로그 추적 등 운영체제 수준의 다양한 기능을 활용하게 되는데, 이는 대부분 리눅스 기반에서 원활하게 수행된다.

리눅스를 선택한 가장 큰 이유는 Windows보다 보안 실습에 훨씬 적합한 구조를 가지고 있기 때문이다. 리눅스는 오픈소스 기반이라 내부 구조가 투명하게 공개되어 있으며, 파일 시스템, 권한 체계, 네트워크 서비스 등 시스템 자원을 세밀하게 제어할 수 있다. 또한 웹 해킹·포렌식·리버스 엔지니어링 등 CTF 분야에서 사용되는 주요 보안 도구는 대부분 리눅스 환경을 기준으로 개발되어 있으며, Bash 스크립트를 활용한 자동화도 간단하여 CTF 문제 환경을 구축하고 관리하기에 매우 효율적이다.

가상머신(VM)을 사용하는 것도 매우 중요한 이유 중 하나이다. VMware 기반 리눅스 환경은 호스트 운영체제(Windows)에 영향을 주지 않으면서 root 권한 실험, 서버 구축, 보안 취약점 재현 등을 자유롭게 수행할 수 있다. 또한 CTF 문제를 제작하다가 시스템 설정을 잘못 변경해도 스냅샷 기능을 통해 즉시 이전 상태로 복구할 수 있어 안전하고 유연한 보안 실습 환경을 제공한다.

설치한 가상머신의 주요 설정

Devices	
Memory	4 GB
Processors	2
Hard Disk (SCSI)	20 GB
CD/DVD 2 (SATA)	Using file C:₩Use...
CD/DVD (SATA)	Using file autoins...
Floppy	Using file autoins...
Network Adapter	NAT
USB Controller	Present
Sound Card	Auto detect
Display	Auto detect

이와 같은 환경을 기반으로, 앞으로의 주차에서는 curl, vim, tar/gzip, netcat, Docker와 같은 리눅스 도구들을 설치하고 각 도구를 활용하여 실제 CTF 문제를 직접 제작하며 보안 개념을 학습할 예정이다.

<설치 과정 요약 >

1. Ubuntu ISO 다운로드

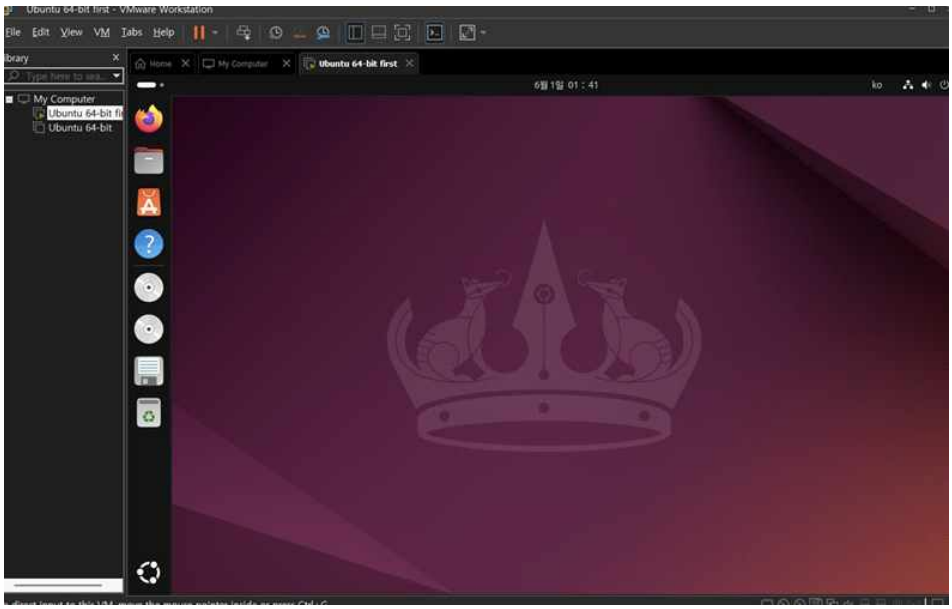
Ubuntu 공식 웹사이트에서 ISO 파일 다운로드 https://ubuntu.com/download

2. VMware 가상 머신 생성

• “Create a New Virtual Machine” 클릭
• ISO 파일 지정 후 Ubuntu 자동 인식
• 메모리 및 CPU, 디스크 용량 설정

3. Ubuntu 설치 진행

• GUI 기반 설치 마법사로 지역/언어/계정 설정
• 설치 완료 후 재시작 및 초기 셋업



2주차 Vim 설치하기

나는 보안 분야에 관심이 있어 이번 리눅스 한 달 살기 프로젝트에서 단순한 명령어 실습을 넘어, 실제로 보안 기법을 체험해보고자 CTF 문제 제작을 함께 진행하기로 했다. 이번 주에는 리눅스 환경에서 가장 기본적이고 강력한 편집기인 Vim을 설치하고, 이를 활용하여 Whitespace Steganography(공백·탭 기반 스테가노그래피) 형태의 CTF 문제를 직접 제작해 보았다. 스테가노그래피 CTF 문제는 보통 평범해 보이는 메시지 속에 비밀 데이터를 숨겨두고, 사용자가 이를 찾아내는 형태로 구성된다.

vim은 vi에서 파생된 확장형 텍스트 편집기로, 리눅스 환경에서 코드 편집이나 설정 파일 수정을 위해 필수적으로 사용된다.
그리고 vim의 특징은 다음과 같다

- 입력 모드, 명령 모드, 비주얼 모드 등 다양한 편집 모드를 제공하여 효율적인 작업이 가능하다.
- 숨겨진 문자 표시 기능(:set list): 공백, 탭, 줄바꿈 등 보이지 않는 문자를 탐지할 수 있어 스테가노그래피 문제 분석에 매우 효과적이다.

이번 실습에서는 특히 :set list 기능을 활용해 눈에 보이지 않는 공백과 탭을 시각적으로 확인하는 경험을 할 수 있었다. 이는 일반 텍스트 에디터에서는 제공되지 않는 Vim만의 강력한 기능 중 하나로, 스테가노그래피 문제 해결에 핵심적인 역할을 했다.

<CTF 문제 제작 과정>
먼저 vim을 설치하고 버전을 확인하였다.

sudo apt install vim : vim 설치
vim –version : 설치 확인 및 버전 확인

그다음 FLAG를 공백과 탭으로 인코딩하기 위한 파이썬 스크립트를 작성하였다.
nano encode_flag.py를 입력하고 이 파일안에 다음 명령어를 입력한다.

```
FLAG = "FLAG{Rabbit_To_Dog_we_Need_BigCarrot}“

# 1) FLAG → 8비트 바이너리 변환
binary = ''.join(format(ord(c), '08b') for c in FLAG)
# 2) 바이너리 → 공백/탭 변환
encoded = ''
for b in binary:
    if b == '0':
        encoded += ' '        # 공백
    else:
```

```
        encoded += '\t'      # 탭
# 3) encoded_flag.txt 파일로 저장
with open("encoded_flag.txt", "w") as f:
    f.write(encoded)
print("encoded_flag.txt 생성 완료!")
```

이 파이썬 코드는 먼저 FLAG 문자열의 각 문자를 8비트 바이너리로 변환한다. 다음으로 이 바이너리 문자열을 순회하며 '0'은 공백문자로 '1'은 tab문자로 치환하여 encoded문자열을 만든다. 최종적으로 이 공백과 탭으로 구성된 문자열을 encoded_flag.txt 파일에 저장한다.

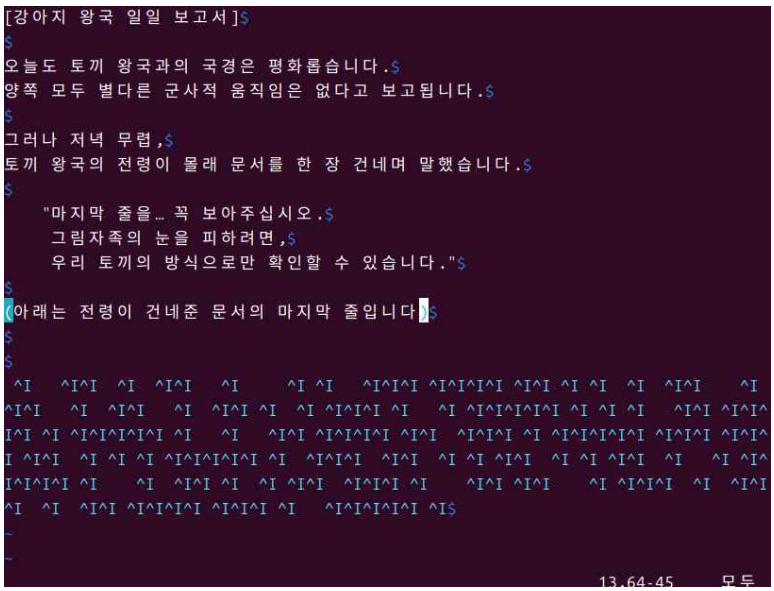
파일을 다 만든 이후 vim 프로그램을 이용해서 problem.txt파일을 만든다.

vim problem.txt

이 파일 위에는 암호화하지 않은 내용을 작성한 후 **:r encoded_flag.txt** 명령어를 입력한다. 이는 특정 파일의 내용을 현재 편집 중인 파일로 불러와 삽입하는 명령어이다. 그러면 눈에 보이지 않는 공백과 탭으로 이루어진 문자열이 현재 파일에 삽입된다.

<CTF 문제 풀이>

먼저 vim problem.txt을 통해서 핵심 파일을 열어본다. 이후 **:set list** 라는 vim 명령어를 활용하여 숨겨진 문자를 시각화한다.



이제 다음 암호문을 해독해야한다.

공백을 '0', 탭을 '1'로 간주하여 데이터를 해석한다. 따라서 마지막 줄의 공백·탭 조합을 순서대로 읽어가며 0과 1로 변환하면 긴 이진 문자열을 얻을 수 있다. 이후 이 이진 문자열을 8비트씩 끊어 ASCII 문자로 디코딩하면 원래 FLAG가 복원된다. 예를 들어, 01000110은 문자 'F', 01001100은 문자 'L'과 같이 변환된다. 이러한 방식으로 전체 데이터를 변환한 결과 최종적으로 다음과 같은 FLAG를 얻을 수 있다.

https://github.com/parkmiso-smile/my-own-CTF_studying-Linux.git

3주차 curl 설치하기

curl은 리눅스에서 가장 널리 사용되는 HTTP 클라이언트 도구로, 웹 서버와 데이터를 주고받기 위한 표준적인 방식으로 활용된다. 터미널에서 직접 요청을 생성할 수 있기 때문에 브라우저를 거치지 않고도 GET, POST, 헤더 변경, 쿠키 주입 등 다양한 형태의 요청 테스트가 가능하다.

이번에 제작한 문제는 웹분야의 CTF 문제이며, 그중에서도 헤더 기반 접근 제어를 다루는 실습이다. curl을 활용해 브라우저가 보낼 수 없는 맞춤형 요청을 만들어 우회하는 과정으로 웹 해킹의 기본 원리를 체험해보고자 한다.

<CTF제작과정>

먼저 curl을 설치한다. 대부분의 리눅스 배포판에는 curl이 기본적으로 설치되어 있지만 만약 설치되어 있지 않다면 다음 명령어로 설치한다.

sudo apt update
sudo apt install curl

이후 서버를 구축한다.

mkdir curl_ctf #실습 디렉토리 생성
cd curl_ctf
nano server.py #flask 서버 코드 만들기

server.py에는 다음 코드를 입력한다.

```
from flask import Flask, request
app = Flask(__name__)
# Flask 애플리케이션 객체를 생성한다. __name__은 현재 모듈의 이름이다.

@app.route('/')
def index():
    ua = request.headers.get("User-Agent")
#현재 요청의 헤더에서 'User-Agent' 값을 추출하여 ua 변수에 저장합니다

    if ua == "KingHacker/1.0":
        return "FLAG{CURL_UA_BYPASS_SUCCESS}“
    # 추출한 User-Agent 값이 "KingHacker/1.0"과 일치하는지 확인하고 일치하면 flag를 출력한다.
    else:
        return "Access Denied: Your browser is not allowed.“

app.run(host="0.0.0.0", port=5000)
```

```
#flask웹 서버를 0.0.0.0호스트와 5000포트에서 실행
```

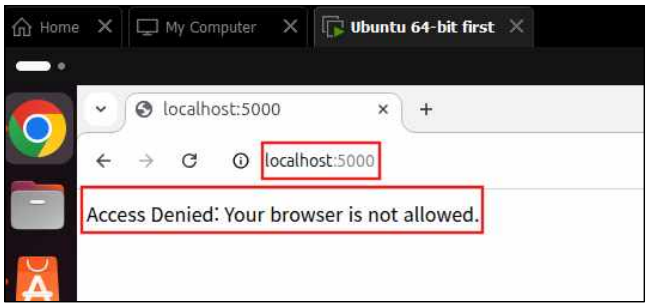
이 코드는 flask 서버 코드를 만드는 것이다. flask는 파이썬 기반으로 작성된 마이크로 웹 프레임워크로 웹 서버나 API를 만들 때 사용한다. 즉 쉽게 쓸 수 있는 웹서버와 같다.
현재 코드에서는 HTTP 요청의 User-Agent 헤더를 확인하여 특정 클라이언트에게만 정보를 제공하고, 그 외의 클라이언트의 접근은 차단하도록 한다.

python3 server.py #서버 실행

```
(venv) newcomputer@newcomputer-VMware-Virtual-Platform:~/curl_ctf$ python3 serve
r.py
 * Serving Flask app 'server'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.110.128:5000
Press CTRL+C to quit
```

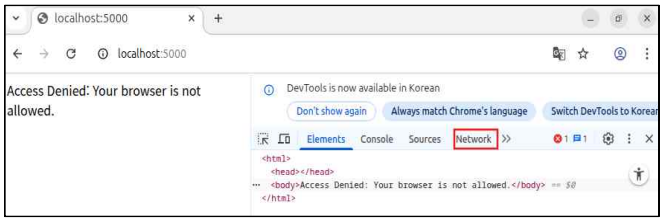
이를 통해 서버가 정상 실행되고 있음을 확인할 수 있다.

<CTF 문제 풀이>
사이트에 접속하면 다음과 같이 뜬다.
우리는 chrome 개발자 도구를 사용하기 위해 chrome으로 접속하였다.

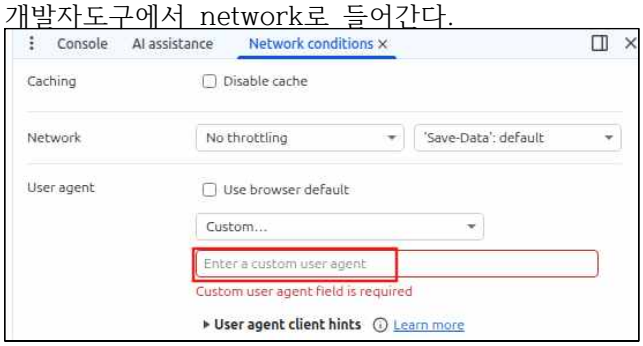


http://localhost:5000/

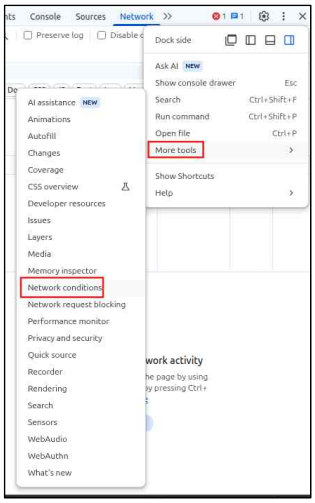
우리는 다음 툴을 이용해서 user agent를 수정할 수 있다.



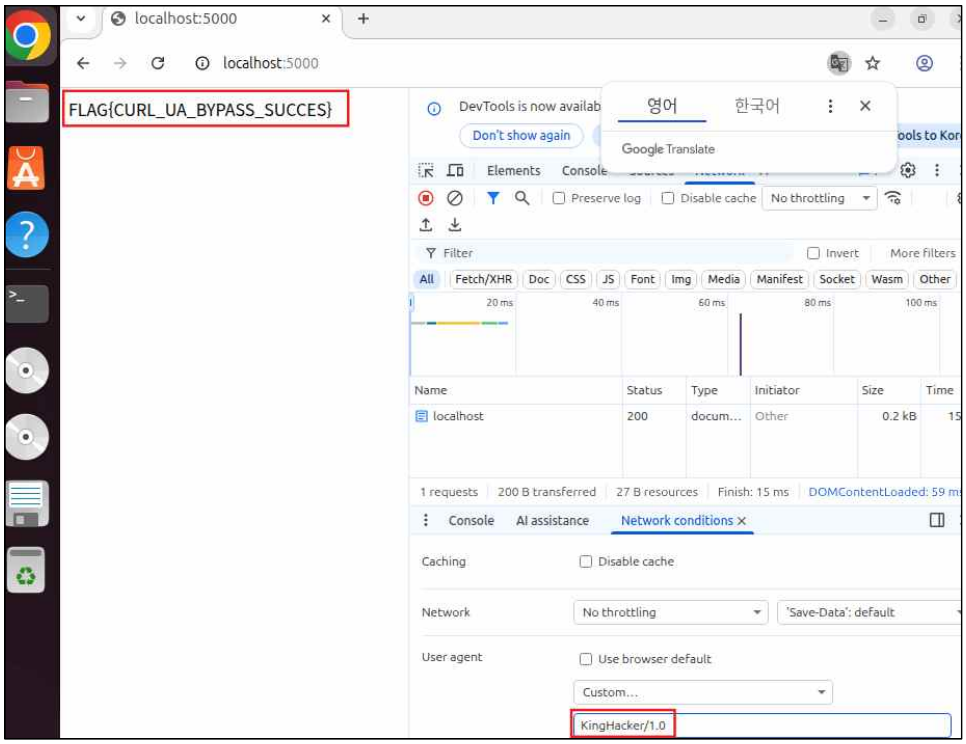
개발자도구에서 network로 들어간다.



user agent를 수정한다



Network condition



FLAG가 나온 것을 확인할 수 있다.

4주차 tar/gzip 설치하기

tar와 gzip은 리눅스에서 가장 널리 사용되는 압축 및 아카이브 도구로, 다수의 파일을 하나의 묶음으로 만들거나 용량을 줄여 백업 및 배포에 활용할 수 있다. 또한 tar와 gzip은 실제 서버 운영 환경과 포렌식 분석 분야에서도 자주 등장하기 때문에, 이를 활용해 문제를 제작하는 과정이 보안 학습에도 큰 될 것이다.

이번에 제작한 문제는 포렌식분야의 CTF 문제로, 압축 파일 내부에 숨겨진 증거 파일을 찾아내는 시나리오이다. tar 아카이브 내부의 디렉토리 구조를 분석하고, gzip으로 압축된 파일을 해제하여 다시 한 번 ZIP 형식의 은닉 파일을 발견·추출하도록 설계했다.

<문제 제작> - https://github.com/parkmiso-smile/Forensics-CTF.git

먼저 디렉토리를 생성한다.

```
mkdir -p case_backup/project/src
mkdir -p case_backup/project/docs
mkdir -p case_backup/project/assets/data/tmp/.cfg
mkdir -p case_backup/logs
```

그 다음으로 파일들을 생성하다.

```
echo "print('server booting...')" > case_backup/project/src/app.py
echo "# helper functions" > case_backup/project/src/helper.py
echo "Confidential Project Documentation" > case_backup/project/docs/readme.md
echo "v1.2 changes logged" > case_backup/project/docs/changes.txt
echo "Server log: OK" > case_backup/logs/server.log
```

그러면 다음과 같은 구조의 파일들이 생성되었다.

```
case_backup/
├── logs/
│   └── server.log
└── project/
├── src/
│   ├── app.py
│   └── helper.py
├── docs/
│       ├── readme.md
│       └── changes.txt
└── assets/
        └── data/
                └── tmp/
                        └── .cfg/
```

여기에 기밀 FLAG파일을 숨겨두자
echo "FLAG{Case_Confidential_Evidence}" > secret_flag.txt
이 파일을 압축파일로 묶고 원래의 기밀 파일을 삭제한다.
zip hidden_payload.zip secret_flag.txt
rm secret_flag.txt

이제 이 파일을 이전에 만든 파일들 사이에 숨겨둔다
mv hidden_payload.zip case_backup/project/assets/data/tmp/.cfg/

이후 전체 폴더를 tar로 묶는다.
tar -cvf case_backup.tar case_backup/
tar을 gzip으로 압축한다.
gzip case_backup.tar

<문제 풀이>

너는 수사관이고, 범죄자의 컴퓨터에서 `case_backup.tar.gz` 라는 압축 파일 1개를 확보했다. 겉보기엔 평범한 프로젝트 백업처럼 보이지만 범죄자는 중요한 증거 파일을 어딘가에 숨겨놓았을 가능성이 있다. 너의 임무: tar 파일 내부 구조를 분석하고, 숨겨진 ZIP 파일과 그 안의 FLAG를 찾아라.

이를 이번 CTF문제의 시나리오라고 두자

tar -tf case_backup.tar.gz #파일 내부에 무엇이 들어있는지 확인한다

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ tar -tf case_backup.tar.gz
case_backup/
case_backup/project/
case_backup/project/assets/
case_backup/project/assets/data/
case_backup/project/assets/data/tmp/
case_backup/project/assets/data/tmp/.cfg/
case_backup/project/assets/data/tmp/.cfg/hidden_payload.zip
case_backup/project/src/
case_backup/project/src/helper.py
case_backup/project/src/app.py
case_backup/project/docs/
case_backup/project/docs/readme.md
case_backup/project/docs/changes.txt
case_backup/logs/
case_backup/logs/server.log
newcomputer@newcomputer-VMware-Virtual-Platform:~$
```

여기서는 flag를 담고 있는 파일을 확인할 수 없다. 즉 secret_flag.txt파일을 찾을 수 없다.
하지만 의심스러운 압축파일이 있으므로 이를 확인한다
tar -xvf case_backup.tar.gz # 파일들을 직접 보기 위해 압축을 해제한다.
cd case_backup/project/assets/data/tmp/.cfg/ #의심되는 경로로 접근하기

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ cd case_backup/project/assets
/data/tmp/.cfg/
newcomputer@newcomputer-VMware-Virtual-Platform:~/case_backup/project/assets/dat
a/tmp/.cfg$ ls
hidden_payload.zip
```

그러면 의심되는 파일을 확인할 수 있다.

unzip hidden_payload.zip #압축 파일 해제하기

```
newcomputer@newcomputer-VMware-Virtual-Platform:~/case_backup/project/assets/dat
a/tmp/.cfg$ unzip hidden_payload.zip
Archive:  hidden_payload.zip
 inflating: secret_flag.txt
```

여기서 발견한 파일을 열어보자

```
newcomputer@newcomputer-VMware-Virtual-Platform:~/case_backup/project/assets/dat
a/tmp/.cfg$ cat secret_flag.txt
FLAG{Case_Confidential_Evidence}
```

flag를 획득할 수있다.

FLAG{Case_Confidential_Evidence}

5주차 vsftpd 설치하기

vsftpd는 very Secure File Transfer Protocol의 약자로 FTP의 보안을 강조한 파일 전송 프로토콜이다. 파일 전송이 주 목적인 FTP서비스는 유닉스 또는 리눅스 계열에서 vsftpd로 사용한다. 여기서 FTP란 인터넷을 통해 파일을 교환할 수 있게 해주는 서비스로 이해할 수 있다.
vsftpd의 핵심적인 보안기능은 다음과 같다.

1. 사용자 격리 기능

vsftpd의 가장 강력한 보안 기능 중 하나로 FTP 사용자가 로그인하면, 해당 사용자의 홈 디렉토리(/home/ftpuser/ 등)를 파일 시스템의 루트 디렉토리(/)처럼 강제로 설정한다.
사용자는 할당된 홈 디렉토리 밖으로 절대 이동할 수 없게 된다. 이를 통해 공격자가 설령 FTP 계정을 획득하더라도 시스템의 민감한 다른 영역(예: /etc, /var/log)에 접근하여 훼손하거나 정보를 탈취하는 것을 방지한다.

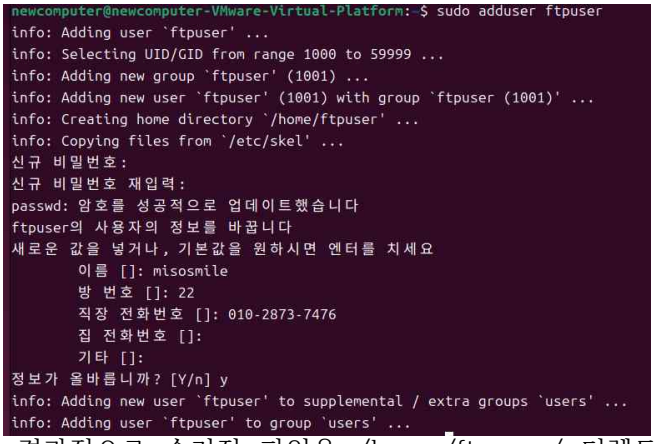
2. 최소 권한 원칙 (Principle of Least Privilege)

vsftpd는 보안에 영향을 미치는 중요 작업 외에는 최소한의 권한만 사용한다. 대부분의 작업은 권한이 낮은 사용자로 수행된다. 만약 서버가 해킹당하더라도, 공격자가 얻을 수 있는 권한이 매우 제한적이어서 시스템 전체에 미치는 피해를 최소화할 수 있다.

이번 CTF문제(ls -a를 통해 숨김 파일을 획득하는 과정)는 파일 시스템과 접근 제어에 대한 두가지 핵심 보안의 중요성을 강조한다. 첫째, 파일 이름에 .(점)을 사용하여 숨기는 은닉기법은 단순 은닉 기법이므로 ls -a 명령에 의해 민간 정보가 즉시 노출될 수 있다. 둘째, 로그인 권한과 파일 접근 권한이 명확히 분리되지 않고 ftpuser 계정에 과도하게 부여되어 있으면 공격적인 합법적인 접속 권한을 악용하여 숨겨진 파일을 포함한 모든 정보를 탈취할 수 있다.

<문제 제작>
sudo apt install vsftpd #vsftpd 설치
이제 FTP서버에 접속할 전용 사용자 계정을 생성한다. 그리고 해당 아이디가 파일을 올리고 내릴 전용 폴더 생성된다.
sudo adduser ftpuser

이제 숨겨진 FLAG 파일을 생성한다.
echo "FLAG{find_hidden_file}" | sudo tee /home/ftpuser/.secret_flag
sudo chown ftpuser:ftpuser /home/ftpuser/.secret_flag
해당 파일은 일반 ls로는 보이지 않는다. 이는 리눅스와 유닉스 계열의 파일 시스템 규칙 때문이다. 리눅스와 유닉스 시스템에서 파일이나 디렉토리 이름이 . (점, Period)으로 시작하면, 해당 항목은 "숨김(Hidden)" 속성을 가지기 때문이다.



```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ sudo adduser ftpuser
info: Adding user `ftpuser' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `ftpuser' (1001) ...
info: Adding new user `ftpuser' (1001) with group `ftpuser (1001)' ...
info: Creating home directory `/home/ftpuser' ...
info: Copying files from `/etc/skel' ...
신규 비밀번호:
신규 비밀번호 재입력:
passwd: 암호를 성공적으로 업데이트했습니다
ftpuser의 사용자의 정보를 바꿉니다
새로운 값을 넣거나, 기본값을 원하시면 엔터를 치세요
        이름 []: misosmile
        방 번호 []: 22
        직장 전화번호 []: 010-2873-7476
        집 전화번호 []:
        기타 []:
정보가 올바릅니까? [Y/n] y
info: Adding new user `ftpuser' to supplemental / extra groups `users' ...
info: Adding user `ftpuser' to group `users' ...
```

이 과정을 통해서 계정을 생성한다.

비밀번호를 생성한 후, 부가 정보를 입력하고, 시스템이 계정을 최종적으로 확정하고 그룹에 추가한다.

결과적으로 숨겨진 파일을 /home/ftpuser/ 디렉토리에 생성하고 관리자(root)가 만든 이 파일의 주인을 ftpuser로 변경하여, ftpuser가 FTP 접속 후 이 파일을 읽거나 수정할 수 있도록 권한을 부여한다.

이제 이 파일을 정상 파일로 위장한다.

sudo mkdir /home/ftpuser/files # ftpuser가 ftp로 접속했을 때 사용할 파일 저장 공간인 폴더 생성

sudo chown ftpuser:ftpuser /home/ftpuser/files # files 폴더의 권한을 ftpuser로 지정

echo "일반 문서입니다" | sudo tee /home/ftpuser/files/readme.txt #files 폴더 안에 readme.txt 파일 생성

sudo chown ftpuser:ftpuser /home/ftpuser/files/readme.txt #파일의 권한을 ftpuser로 바꿔줌

이후 해킹을 위해 로그에 “공격자 흔적”을 남길 수 있다. 그러나 이는 진짜 서버가 자동으로 기록한 로그가 아니라, 실습을 위해 직접 로그 파일에 삽입한 가짜 로그라는 것을 유의하자.

<문제 풀이>

시나리오

당신은 보안 연구원으로서, 외부 공격자에 의해 유출된 것으로 보이는 로그 파일을 분석하고 있다. 로그 파일에는 다음과 같은 의미심장한 흔적이 남아 있었습니다.

로그 흔적 분석: Mon Nov 24 20:11:22 2025 [pid 2] [ftpuser] OK UPLOAD: "/home/ftpuser/.secret_flag“

이 로그는 ftpuser라는 계정이 .secret_flag라는 이름의 숨겨진 파일을 특정 시점에 서버에 업로드했다는 기록입니다.

FTP 서비스가 활성화되어 있고, 로그에 기록된 ftpuser의 계정 정보를 이미 알고 있다는 가정 하에 (약한 비밀번호나 다른 취약점을 통해 획득했다고 가정), FTP 클라이언트를 사용하여 서버에 접속하고 이 숨겨진 파일을 찾아 다운로드함으로써 데이터 유출 흔적의 실체를 확인해야 합니다.

FTP서버는 파일 접근 권한이 FTP 로그인을 통해서만 가능하다.

그러므로 먼저 FTP에 접속한다.

ftp 127.0.0.1

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ ftp 127.0.0.1
Connected to 127.0.0.1.
220 (vsFTPd 3.0.5)
Name (127.0.0.1:newcomputer):
```

이렇게 뜨면 현재 FTP 서버에 성공적으로 연결된 것이다. 이후 서버가 사용자의 이름을 묻고 있다. 그러면 아까 생성한 사용자 계정이름인 ftpuser을 입력한다. 이후 비밀번호도 입력하면 정상적으로 로그인된다.

ls #/home/ftpuser의 파일 목록을 보여준다. 이때 **.secret_flag** 파일은 보이지 않는다.

ls -a **#**이 명령어를 입력하면 숨김 파일까지 확인 가능하다.

```
ftp> ls
229 Entering Extended Passive Mode (|||36596|)
150 Here comes the directory listing.
drwxr-xr-x    2 1001     1001         4096 Nov 24 20:13 files
226 Directory send OK.
ftp> ls -a
229 Entering Extended Passive Mode (|||6887|)
150 Here comes the directory listing.
drwxr-x---    3 1001     1001         4096 Nov 24 20:50 .
drwxr-xr-x    4 0        0            4096 Nov 24 20:03 ..
-rw-r--r--    1 1001     1001          220 Nov 24 20:03 .bash_logout
-rw-r--r--    1 1001     1001         3771 Nov 24 20:03 .bashrc
-rw-r--r--    1 1001     1001          807 Nov 24 20:03 .profile
-rw-r--r--    1 1001     1001           23 Nov 24 20:50 .secret_flag
drwxr-xr-x    2 1001     1001         4096 Nov 24 20:13 files
226 Directory send OK.
```

이제 숨겨진 파일 .secret_flag가 있음을 확인할 수 있다. 이후에 우리는 이 숨겨진 파일을 get명령으로 다운로드 한다.

ftp> **get .secret_flag**

exit # ftp 빠져나가기

cat .secret_flag # 로컬에서 파일 내용 확인하기

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ cat .secret_flag
FLAG{find_hidden_file}
```

우리가 flag를 획득했음을 알 수 있다.

6주차 netcat(nc)/nmap 설치하기

Netcat(nc)은 TCP/IP 프로토콜을 사용하여 네트워크 연결을 통해 데이터를 읽고 쓰는 명령줄 유틸리티이다. 네트워크 디버깅, 포트 스캔, 파일 전송 등을 위해 사용되는 백엔드 도구이다.

Nmap은 네트워크 탐색 및 보안 감사를 위해 도구이다. 컴퓨터 네트워크의 호스트, 서비스 및 취약점을 검색하도록 설계되었다. Nmap의 몇 가지 주요 기능은 다음과 같다

(1) ICMP(인터넷 제어 메시지 프로토콜) 핑, ARP(주소 확인 프로토콜) 요청, TCP SYN 스캔 등과 같은 호스트를 확인할 수 있다.

(2) 대상 호스트에서 열린 포트를 스캐닝하고 식별할 수 있다.

(3) Nmap은 열려 있는 포트를 조사하여 해당 포트에서 실행되는 서비스 버전을 확인할 수 있다.

초기 세팅을 위해 먼저 netcat과 nmap를 설치할 것이다.

sudo apt install netcat-openbsd # netcat설치

sudo apt install nmap #Nmap 설치

<문제 제작>

mkdir netcat-ctf

cd netcat-ctf

mkdir challenge

mkdir server

cd server

echo "FLAG{Scan_To_Find_The_Secret_Port}" > flag.txt

cat flag.txt | nc -lvp 23456

이 명령어는 다음과 같은 순서로 실행된다.

1. 리스너 대기: nc가 23456 포트에서 외부 연결을 기다린다.
2. 연결 수락: 외부의 다른 컴퓨터나 클라이언트가 이 포트로 접속을 시도한다.
3. 데이터 전송: 연결이 수립되는 즉시, cat flag.txt 명령을 통해 파이프(|)로 전달된 flag.txt의 내용이 네트워크 연결을 통해 접속한 클라이언트에게 전송된다.
4. 세션 종료: 일반적으로 nc는 파일 내용을 모두 전송하거나 클라이언트가 연결을 끊으면 세션을 종료하고 대기 상태를 해제한다.

즉, 이 명령어는 flag.txt의 내용을 담고 있는 일회용 네트워크 서버를 23456 포트에 구축하는 것과 같다.

이 문제와 관련된 설명 파일을 추가한다.

cd challenge

nano intro.md

Week 5 CTF – Hidden Port Challenge
서버는 어떤 포트 하나에 FLAG를 숨겨두었다. 포트 번호는 공개되지 않았으며, 직접 스캔해야 한다. 해야 할 일 1. 서버에서 열린 포트를 스캔한다. 2. 특정 포트를 찾는다. 3. netcat(nc)으로 포트에 접속한다. 4. FLAG를 획득한다. <힌트> 포트 스캔 명령어: nmap -p- <server_ip>

<문제 풀이>

1. 먼저 포트를 스캔한다.

nmap -p- 127.0.0.1

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ nmap -p- 127.0.0.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-11-24 22:58 KST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00011s latency).
Not shown: 65529 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
631/tcp   open  ipp
3306/tcp  open  mysql
23456/tcp open  aequus
33060/tcp open  mysqlx

Nmap done: 1 IP address (1 host up) scanned in 2.34 seconds
```

그러면 이런 식으로 결과가 뜨는 것을 확인할 수 있다. 여기서 우리가 특정 포트에 접속해야 한다. 이때 우리는 일반 서비스가 아닌 포트, 즉 23456 포트를 의심할 수 있다. 그러므로 해당 포트에 접속한다.

2. 포트 접속

nc 127.0.0.1 23456

이는 현재 실행 중인 컴퓨터(127.0.0.1 또는 로컬호스트)의 23456번 포트로 네트워크 연결을 시도하고 통신을 시작하는 것이다.

3. 플래그 획득

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ nc 127.0.0.1 23456
FLAG{Scan_To_Find_The_Secret_Port}
```

해당 포트에 접속하여 플래그를 얻은 것을 확인할 수 있다.

7주차 mariaDB 설치하기

이번 실습은 maria DB프로그램을 이용해서 CTF를 제작하였다. MariaDB는 무료 오픈소스 관계형 데이터베이스 관리 시스템이다. MySQL과 구조적으로 거의 동일하지만 더 가볍고 설치가 간편한 MariaDB를 사용하였다. MySQL은 Oracle이 관리하는 상업적 성격이 있는 반면, MariaDB는 100% 오픈소스로서 교육·실습 환경에서 제약 없이 활용할 수 있다.

<문제 제작>

CTF 시나리오: 쇼핑몰 주문 조작 사건 2024년 10월 3일, 쇼핑몰의 내부 데이터베이스에서 수상한 주문 결제 기록이 발견되었다. 한 사용자가 고가의 물건을 주문했음에도 해당 주문의 결제 금액이 0원으로 기록되어 있었다. 관리 로그(logs 테이블)를 확인한 결과, 일부 로그가 삭제된 듯한 흔적도 발견되었다. 주문을 조작한 사람의 이름을 알아내라.

먼저 mariaDB 서버를 설치한다.

sudo apt install mariadb-server #마리아 DB설치
sudo systemctl start mariadb # MariaDB 서비스를 시작
sudo systemctl enable mariadb # 부팅 시 MariaDB 자동 시작 설정
sudo mysql_secure_installation # 초기 보안 설정 스크립트 실행 (비밀번호 설정 등)
sudo mysql -u root -p # 루트 사용자로 MariaDB에 접속

CTF에서 사용 될 데이터베이스를 생성한다.

```
CREATE DATABASE forensicdb;
USE forensicdb;
```

이제 데이터베이스 안에 테이블을 만든다. 제작할 테이블은 총 4개로 users,orders, payment, logs 테이블이다.

```
CREATE TABLE users(
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50),
  email VARCHAR(100),
  password_hash VARCHAR(255)
);
```

users 테이블

```
CREATE TABLE orders(
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  product_name VARCHAR(100),
  price INT,
  status VARCHAR(20),
  timestamp DATETIME
);
```

orders 테이블

```
CREATE TABLE payments(
  id INT PRIMARY KEY AUTO_INCREMENT,
  order_id INT,
  amount INT,
  method VARCHAR(20),
  timestamp DATETIME
);
```

payments 테이블

```
CREATE TABLE logs(
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  action VARCHAR(255),
  timestamp DATETIME
);
```

logs 테이블

이제 각 테이블에 정상 데이터들을 삽입한다.

```
#정상 유저 데이터
INSERT INTO users(username, email, password_hash) VALUES
('alice', 'alice@example.com', 'xxx'),
('bob', 'bob@example.com', 'xxx'),
('charlie', 'charlie@example.com', 'xxx'),
('attacker', 'hack@hackmail.com', 'xxx');
# 정상 주문 데이터
INSERT INTO orders(user_id, product_name, price, status, timestamp) VALUES
(1, 'Keyboard', 45000, 'paid', '2024-10-01 10:10:00'),
(2, 'Mouse', 25000, 'paid', '2024-10-01 11:00:00'),
(3, 'Monitor', 200000, 'paid', '2024-10-02 09:00:00');
# 정상 결제 데이터
INSERT INTO payments(order_id, amount, method, timestamp) VALUES
(1, 45000, 'card', '2024-10-01 10:11:00'),
(2, 25000, 'card', '2024-10-01 11:01:00'),
(3, 200000, 'bank', '2024-10-02 09:01:00');
# 정상 로그
INSERT INTO logs(user_id, action, timestamp) VALUES
(1, 'ORDER_CREATE:1', '2024-10-01 10:10:01'),
(2, 'ORDER_CREATE:2', '2024-10-01 11:00:01'),
(3, 'ORDER_CREATE:3', '2024-10-02 09:00:01');
```

이후에 공격자가 조작한 데이터를 넣는다.
먼저 주문 데이터를 생성한다.
INSERT INTO orders(user_id, product_name, price, status, timestamp) VALUES
(4, 'Gaming Chair', 300000, 'paid', '2024-10-03 01:00:00');

공격자가 조작한 결제 정보를 삽입한다.
여기서 정상결제라면 300,000이어야 하는데 amount=0으로 두는 것이다.
INSERT INTO payments(order_id, amount, method, timestamp) VALUES
(4, 0, NULL, '2024-10-03 01:00:01');

다음으로 로그를 만들어줄 것이다. 이 로그는 공격자가 자신의 기록을 지우려고 했음을 알려주며 우리가 만든 CTF문제의 힌트가 된다.

INSERT INTO logs(user_id, action, timestamp) VALUES (4, 'ORDER_CREATE:4', '2024-10-03 01:00:01'),(0, 'SYSTEM_ALERT: Incomplete log chain detected for user_id=4', '2024-10-03 01:00:02');

이제 해당 데이터베이스 파일을 가지고 온다.

sudo mysqldump forensicdb > shopping_fraud.sql

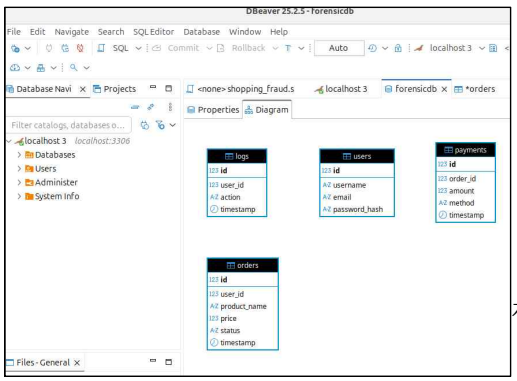
<문제 풀이 방법>

데이터베이스를 더 편리하게 보기 위해 DBeaver를 설치한다.

DBeaver는 프로그램을 실행하고 MariaDB 서버의 IP 주소, 사용자명, 비밀번호만 입력하면 바로 접속하여 데이터베이스를 관리할 수 있다.

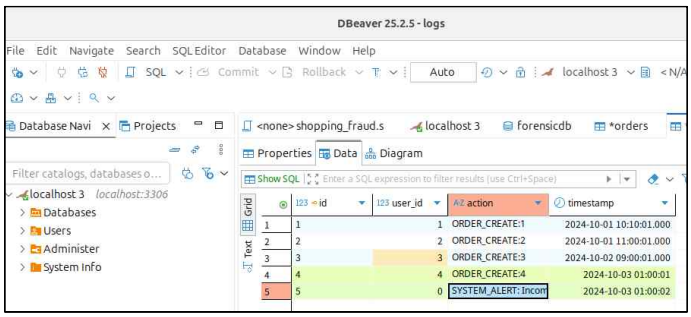
https://dbeaver.io/download/ DBeaver는 해당 사이트에서 다운로드 받는다.

해당 프로그램에서 shopping_fraud.sql파일을 열면 다음과 같이 확인 가능하다.



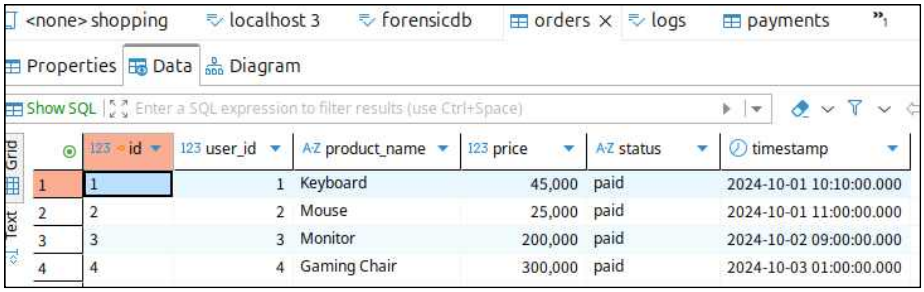
전체 데이터베이스의 테이블들을 확인 가능하다.

먼저 logs테이블에서 수상한 로그를 확인한다.



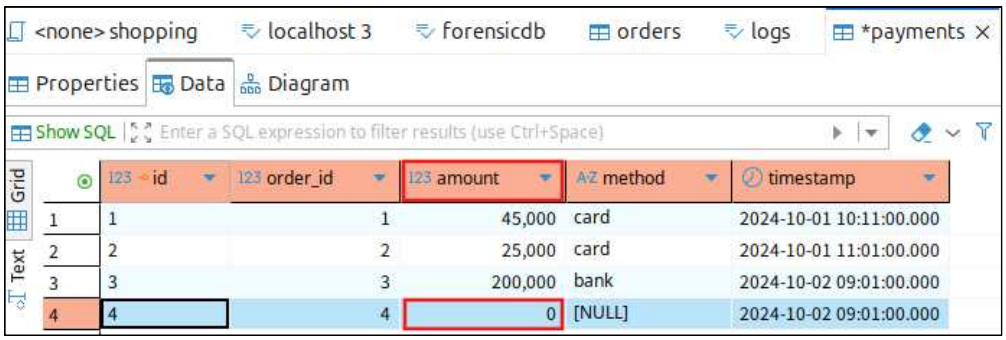
SYSTEM_ALERT: Incomplete log chain detected for user_id=4 라는 로그를 확인할 수 있다. 즉 user_id=4의 행동이 의심스럽다.

orders 테이블에서 user_id=4의 주문을 확인해보면, 30만 원짜리 Gaming Chair가 'paid' 상태로 등록되어 있다.



	id	user_id	product_name	price	status	timestamp
1	1	1	Keyboard	45,000	paid	2024-10-01 10:10:00.000
2	2	2	Mouse	25,000	paid	2024-10-01 11:00:00.000
3	3	3	Monitor	200,000	paid	2024-10-02 09:00:00.000
4	4	4	Gaming Chair	300,000	paid	2024-10-03 01:00:00.000

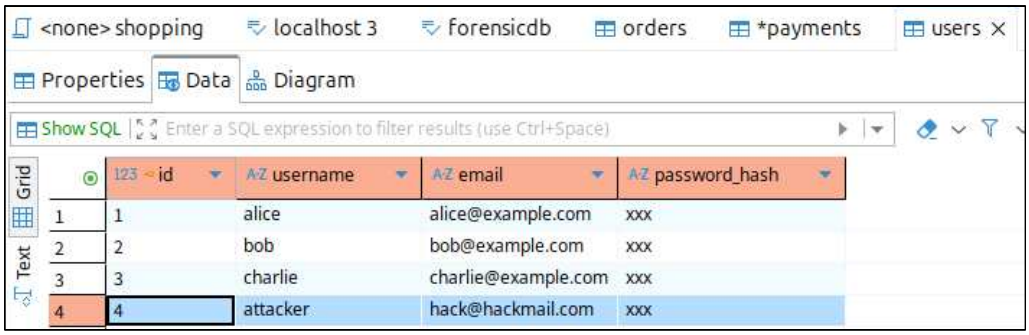
겉보기에는 정상 결제처럼 보이지만, payments 테이블에서 order_id=4의 결제 내역을 조회하면 amount가 0원으로 되어 있다.



	id	order_id	amount	method	timestamp
1	1	1	45,000	card	2024-10-01 10:11:00.000
2	2	2	25,000	card	2024-10-01 11:01:00.000
3	3	3	200,000	bank	2024-10-02 09:01:00.000
4	4	4	0	[NULL]	2024-10-02 09:01:00.000

즉, 정상 결제라면 300,000원이 기록되어야 하지만 공격자가 결제 금액을 0으로 조작한 것이다.

이걸 주문한 사람의 id를 보면, 즉 order_id를 확인하면 4인 것을 알 수 있다.
마지막으로 users id에서 id=4의 정보를 확인한다.



	id	username	email	password_hash
1	1	alice	alice@example.com	xxx
2	2	bob	bob@example.com	xxx
3	3	charlie	charlie@example.com	xxx
4	4	attacker	hack@hackmail.com	xxx

8주차 Bind9 설치하기

BIND는 기본적으로 DNS 서버의 역할을 수행하며, 사용자 친화적인 도메인 이름을 컴퓨터가 이해하는 IP 주소로 변환하는 기능을 한다.

DNS 조작 공격을 이해하고 방어 관점에서 사고하는 능력을 키우기 위해 CTF룰 제작했다. 이를 통해 Bind9가 어떻게 도메인 정보를 관리하고, 작은 레코드 변경만으로도 서비스가 공격자 의도대로 변조될 수 있음을 직접 확인해보고, 이를 통해 DNS 하이재킹이 실제로 어떤 방식으로 이뤄지는지를 학습하고자 한다. 또한 zone 파일 분석 및 레코드 추적 과정을 통해 보안 사고 발생 시 문제 지점을 식별하는 방법도 익힐 수 있다.

<문제 제작>

먼저 Bind9를 설치한다.

sudo apt install bind9 bind9-utils -y

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ ls /etc/bind
bind.keys  db.255    named.conf                named.conf.options
db.0       db.empty  named.conf.default-zones  rndc.key
db.127     db.local  named.conf.local          zones.rfc1918
```

bind9의 기본 파일을 보면 위와 같다.

-주요 파일들

파일/폴더	역할
/etc/bind/named.conf	메인 설정 파일
/etc/bind/named.conf.options	DNS 옵션 이곳에서 설정
/etc/bind/named.conf.local	사용자가 추가하는 zone 설정
/etc/bind/zones/	zone 파일들 저장(우리가 만들 폴더)

bind9는 /etc/bind/ 디렉터리 내에 설정 파일들을 가지고 있지만, 실제로 도메인 이름과 IP 주소의 매핑 정보가 담긴 핵심 데이터 파일(Zone)파일은 관리자가 별도의 위치에 두도록 권장한다.

sudo mkdir /etc/bind/zones # zone파일 생성

sudo nano /etc/bind/zones/db.club-khu.kr

이제 이 안에 bind9 DNS서버가 club-khu.kr 도메인에 대해 권한 있는 응답을 제공하기 위해 사용하는, 도메인 이름과 ip주소등의 정보를 담고 있는 핵심 데이터 파일이다.

```
$TTL    600
@       IN      SOA     ns1.club-khu.kr. admin.club-khu.kr. (
                        2024112201 ; Serial
                        3600       ; Refresh
                        1800       ; Retry
                        604800     ; Expire
                        86400 )    ; Minimum TTL

; name servers
        IN      NS      ns1.club-khu.kr.
```

```
; 정상 A레코드 (원래)
ns1             IN      A       203.0.113.10
www             IN      A       203.0.113.10

; 공격자가 추가한 악성 레코드
; don't look here :)
; flag is definitely not in a TXT record
@               IN      A       198.51.100.66

; 숨겨진 FLAG 서브도메인 (브라우저 접근용)
secret          IN      A       127.0.0.1
```

secret.club-khu.kr은 DNS에서 127.0.0.1로 변환되도록 설정되어 있다. 따라서 브라우저가 secret.club-khu.kr에 접속하면 실제로는 로컬 서버(127.0.0.1)로 요청이 전달된다. 이때 서버는 Host: secret.club-khu.kr 헤더를 보고 별도의 페이지(Python 서버)를 응답한다. 반면 127.0.0.1로 직접 접속하면 Host가 달라 다른 화면이 나타난다.

이제 우리가 만든 zone파일을 bind9설정에 등록해야 DNS서버가 사용한다.

sudo nano /etc/bind/named.conf.local

```
zone "club-khu.kr" {
    type master;
    file "/etc/bind/zones/db.club-khu.kr";
};
```

해당 내용을 파일에 추가한다.

sudo chown -R bind:bind /etc/bind/zones #zone파일 권한을 bind 사용자에게 부여

지금 우분투는 기본적으로 공유기 DNS(192.168.x.x)를 사용하고 있다. 그래서 secret.club-khu.kr 접속해도 Bind9에게 물어보지 않으므로 우리가 만든 zone이 동작하려면 우분투가 DNS 질의를 127.0.0.1(Bind9 서버)에게 보내도록 바꿔야 한다.

sudo nano /etc/resolv.conf

```
nameserver 127.0.0.53
options edns0 trust-ad
search localdomain
```

여기서 nameserver 부분을 127.0.0.1 로 바꿔야 한다. 이제 크롬/파이어폭스에서 입력하는 모든 웹사이트 DNS lookup이 내 PC의 Bind9 서버를 거치게 된다.

브라우저가 secret.club-khu.kr을 입력하면 내 pc로 이동하므로 내 pc에서 웹 서버를 띄워야 한다.

echo "FLAG{DNS_HIJACKING_SUCCESS}" > index.html #웹 페이지 생성

sudo python3 -m http.server 80 # 웹 서버 실행

<문제 풀이>

경희대학교 동아리 연합 홈페이지 club-khu.kr에서 학생들이 접속하면 정상 사이트가 뜨지 않고,수상한 피싱 페이지로 이동하는 현상이 발생했다. 웹 서버에는 문제 없었지만, DNS 서버(BIND9)의 zone 파일에서 누군가 고의로 레코드를 변경한 흔적이 발견되었다. 이것을 분석해서 FLAG를 획득하여라

cat /etc/bind/named.conf.local # 어떤 zone파일이 로드 되는지를 확인한다.

```
zone "club-khu.kr" {
    type master;
    file "/etc/bind/zones/db.club-khu.kr";
};
```

club-khu.kr에 대한 zone 파일이 /etc/bind/zones/db.club-khu.kr 이라는 정보 확보한다

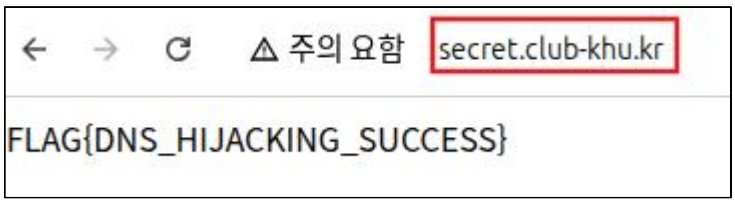
cat /etc/bind/zones/db.club-khu.kr

이 파일을 분석해서 다음과 같은 사실을 알아낸다.

레코드	유형	데이터	분석결과
www	IN A	203.0.113.10	**정상 레코드:** www.club-khu.kr은 원래의 동아리 홈페이지 서버 IP를 가리킨다
@	IN A	**127.0.0.2**	**조작된 레코드:** @는 도메인 자체(club-khu.kr)를 의미한다. 공격자는 이를 악성 IP인 127.0.0.2로 변경하여, 사용자가 메인 도메인으로 접속 시 **피싱 페이지로 강제 리다이렉션**되도록 설정했다
secret	IN A	**127.0.0.1**	**숨겨진 레코드:** secret.club-khu.kr 서브 도메인을 로컬 루프백 주소인 127.0.0.1로 설정했다. 이는 **FLAG가 숨겨진 서버의 위치**를 나타내는 가장 중요한 단서입니다.

이를 바탕으로 http://club-khu.kr 에 접속하여 피싱페이지에 직접 접속해본다.

다음으로 http://secret.club-khu.kr 비밀 서버에 접속하여 FLAG를 획득한다.



9주차 Steghid 설치하기

Steghide는 이미지·오디오 파일 내부에 데이터를 숨기고 추출할 수 있는 스테가노그래피 도구로, 비밀번호 기반으로 안전하게 비밀 정보를 삽입·복원할 수 있다. 이번 주에는 이 프로그램을 이용해서 CTF를 제작해볼 것이다.

<문제제작>

sudo apt install steghide #Steghid 설치

먼저 이미지를 하나 준비한다.

poster.jpg

다음으로 숨길 메시지 파일을 만든다

echo "FLAG{STEGHIDE_SECRET_DISCOVERED}" > secret.txt

이제 페스워드를 설정한 후 이미지에 파일을 삽입한다.

steghide embed -cf poster.jpg -ef secret.txt -p misomiso1004

패스워드를 예를 들어 misomiso1004로 한 것이다.

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ steghide embed -cf poster.jpg
 -ef secret.txt -p misomiso1004
embedding "secret.txt" in "poster.jpg"... done
```

비밀번호를 숨기기 위해 추가적인 도구를 설치한다. exiftool은 이미지 메타데이터를 분석하기 위해 사용되는 도구이다.

sudo apt install steghide exiftool

exiftool -Comment="password = misomiso1004" poster.jpg # 비밀번호 관련 힌트를 그림 안에 숨겨 둔다

<문제 풀이>

경희대학교 프로그래밍 동아리 홍보 포스터에 이상한 점이 있다! 포스터 해상도나 내용은 평범한데,이미지 파일 크기가 유난히 크다는 신고가 접수되었다. 보안 관리자는 “누군가 포스터 이미지 내부에 비밀 메시지를 숨겼다”고 의심하고 문제 해결을 요청했다. 이미지 내부에서 숨겨진 데이터(FLAG)를 추출하여 공격자가 남긴 메시지를 밝혀내야 한다.

먼저 포스터에 데이터가 숨겨져 있는지 확인한다.

steghide info poster.jpg

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ steghide info poster.jpg
"poster.jpg":
  format: jpeg
  capacity: 3.4 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase:
steghide: could not extract any data with that passphrase!
```

하지만 우리는 아직 암호를 모르는 상태이므로 열어 볼 수가 없다.

다음으로 우리는 이미지의 메타데이터를 분석한다,

exiftool poster.jpg

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ exiftool poster.jpg
ExifTool Version Number         : 12.76
File Name                       : poster.jpg
Directory                       : .
File Size                       : 68 kB
File Modification Date/Time     : 2025:11:25 23:09:11+09:00
File Access Date/Time           : 2025:11:25 23:09:17+09:00
File Inode Change Date/Time     : 2025:11:25 23:09:11+09:00
File Permissions                : -rw-rw-r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Comment                         : password = misomiso1004
Image Width                     : 736
Image Height                    : 829
```

여기서 우리는 비밀번호에 관련된 힌트를 얻었다.

다시 **steghide info poster.jpg를** 이용해서 숨겨진 데이터가 있는지 확인한다

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ steghide info poster.jpg
"poster.jpg":
  format: jpeg
  capacity: 3.4 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "secret.txt":
    size: 33.0 Byte
    encrypted: rijndael-128, cbc
    compressed: yes
```

숨겨진 파일이 있음을 확인했다.

steghide extract -sf poster.jpg -p misomiso1004 #이미지에서 파일 추출

cat secret.txt # 파일 열어보기

이제 우리는 flag를 획득했다

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ cat secret.txt
FLAG{STEGHIDE_SECRET_DISCOVERED}
```

10주차 ffmpeg 설치하기

FFmpeg는 오디오 및 비디오 파일을 처리하는 데 사용되는 포괄적인 명령줄 도구 모음이자 라이브러리이다. 미디어 파일 형식 변환, 인코딩, 디코딩, 스트리밍, 필터링 등 거의 모든 미디어 관련 작업을 수행있다.

<문제 제작>

sudo apt install ffmpeg -y #ffmpeg 설치

다음으로 TTS를 설치한다. TTS는 텍스트를 사람의 목소리와 유사한 음성으로 변환하여 출력하는 기술이며. espeak는 다양한 언어를 지원하는 오픈소스 TTS 프로그램이다.

sudo apt install espeak -y #espeak(TTS)설치

echo "FLAG{ADVANCED_REVERSE_AUDIO}" > flag.txt #FLAG 파일 준비

이제 해당 파일을 TTS로 FLAG를 포함한 음성을 생성한다.

espeak "This is the encrypted broadcast. The flag is ADVANCED_REVERSE_AUDIO." -w original.wav

이렇게 생성된 파일은 wav파일이다. wav는 오데오 데이터를 저장하는 가장 일반적인 컨테이너 형식 중 하나이다 wav파일은 단순히 소리 데이터만 담고 있는 것이 아니라, 이 소리 데이터가 어떤 형식으로 되어있는지 알려주는 메타데이터가 헤더에 있다.그러나 우리는 암호화를 위해서는 순수한 오디오만 가지고 와야한다. 그래서 우리는 WAV를 헤더 없는 RAW 데이터로 바꾼다

ffmpeg -i original.wav -f s16le -acodec pcm_s16le raw.pcm

이제 이 파일을 암호화한다.

openssl enc -aes-256-cbc -salt -in raw.pcm -out encrypted.pcm -k parkmiso0422

ffmpeg -f s16le -ar 44100 -ac 1 -i encrypted.pcm encrypted.wav #PCM 형식을 암호화한 것을 다시 WAV 컨테이너에 넣어 정상적인 wav 파일처럼 만든다.

ffmpeg -i encrypted.wav -af "areverse" challenge.wav #이 파일을 뒤집어서 역재생 되도록 한다.

aplay challenge.wav #들어보면 이상한 기계음이 들린다.

<문제 풀이>

학교 방송반 서버에 저장된 안내 방송 음성이 공격자에 의해 AES-256으로 암호화된 뒤 거꾸로(reverse) 저장된 상태로 유출되었다. 관리자 조사 결과, 공격자는 암호화 비밀번호를 “방송반 회장 이름 + 생년월일” 조합으로 만든다는 것이 로그에서 확인된다. 이 안내 음성을 복원하여 flag를 획득해보시오.

문제와 함께 제공된 방송반 홍보 포스터(이미지)를 살펴보면 동아리 회장 이름: 박미소 → parkmiso 이고 회장 이메일: miso0422@…임을 통해 이메일 아이디에 포함된 “0422”는 회장의 생년월일임을 유추할 수 있다.

따라서 암호화 비밀번호는 parkmiso0422라고 알아낸다. 알아낸 비밀번호를 바탕으로 음성파일을 복원해 본다

ffmpeg -i challenge.wav -af "areverse" reversed.wav # reverse 제거

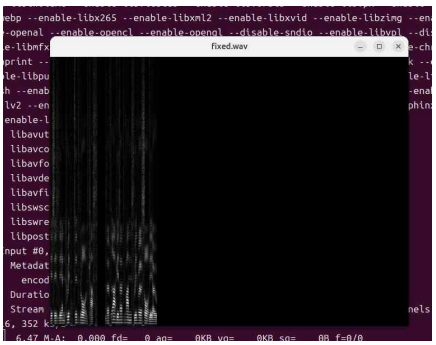
ffmpeg -i reversed.wav -f s16le -acodec pcm_s16le reversed_raw.pcm # wav를 pcm으로 전환하기

openssl enc -d -aes-256-cbc -in reversed_raw.pcm -out decrypted.pcm -k parkmiso0422 #AES를 복호화한다

ffmpeg -f s16le -ar 22050 -ac 1 -i decrypted.pcm fixed.wav #다시 pcm에서 wav로 복원한다.

이제 다시 실행하면 정상 음성이 들린다

ffplay fixed.wav # 음성 실행



fixed.wav 가 실행되는 것을 볼 수 있다.

들어보면 **This is the encrypted broadcast. The flag is ADVANCED_REVERSE_AUDIO** 라는 음성을 들을 수 있다. 그래서 우리는 advanced_reverse_audio라는 flag를 획득했다.

11주차 Hydra/OpenSSH 설치하기

이번 주차에서는 Hydra와 OpenSSH를 이용한 비밀번호 무차별 대입 공격 실습을 진행하였다.

우선, Hydra는 비밀번호 후보를 자동으로 빠르게 입력해보는 공격 도구이다. 사람이 직접 비밀번호를 하나씩 입력하면 시간이 매우 오래 걸리지만, Hydra는 준비된 비밀번호 리스트를 바탕으로 매우 빠르게 로그인 시도를 반복한다. 이러한 방식은 실제 해커들이 약한 비밀번호를 찾기 위해 많이 사용하는 기법이며, 특히 서버나 웹 서비스의 로그인 페이지를 공격할 때 자주 활용된다.

다음으로, OpenSSH는 서버에 원격으로 접속할 수 있도록 해주는 프로그램이다. 예전의 telnet이나 FTP와 같은 오래된 방식은 사용자가 입력한 비밀번호나 명령어가 평문(암호화되지 않은 형태)으로 전송되었기 때문에 중간에서 쉽게 도청될 위험이 있었다. 그러나 OpenSSH는 이러한 데이터를 암호화하여 전송하기 때문에 더욱 안전하게 원격 접속을 할 수 있게 해준다.

이번 실습에서는 가상 머신에 OpenSSH 서버를 활성화한 뒤, 쉬운 비밀번호를 가진 테스트 계정을 만들어 Hydra로 공격을 수행하였다. 그 결과, SSH 서버에 약한 비밀번호가 설정되어 있을 경우 Hydra가 매우 빠르게 비밀번호를 알아내고 계정을 침투할 수 있다는 것을 직접 확인할 수 있었다. 이를 통해 약한 비밀번호의 위험성과 서버 보안 설정의 중요성을 이해할 수 있었다.

이제 실습을 진행해보자

먼저 Hydra와 OpenSSH를 설치한다.

sudo apt install hydra -y #hydra 설치

sudo apt install openssh-server -y # openssh설치

SSH 서버 자동 실행 설정

sudo systemctl enable ssh

sudo systemctl start ssh

Hydra는 '맞지 않는 비밀번호'를 여러 개 넣어보는 실험이다. 그러려면 테스트용 계정이 필요하다

sudo adduser hydratest # Linux 시스템에서 새로운 계정 생성

여기서 우리는 비밀번호를 miso1234로 설정해보겠다.

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ sudo adduser hydratest
info: Adding user `hydratest' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hydratest' (1002) ...
info: Adding new user `hydratest' (1002) with group `hydratest (1002)' ...
info: Creating home directory `/home/hydratest' ...
info: Copying files from `/etc/skel' ...
신규 비밀번호:
신규 비밀번호 재입력:
passwd: 암호를 성공적으로 업데이트했습니다
hydratest의 사용자의 정보를 바꿉니다
새로운 값을 넣거나, 기본값을 원하시면 엔터를 치세요
        이름 []: misocute
        방 번호 []: 88
        직장 전화번호 []:
        집 전화번호 []:
        기타 []:
정보가 올바릅니까? [Y/n] y
info: Adding new user `hydratest' to supplemental / extra groups `users' ...
info: Adding user `hydratest' to group `users' ...
```

우리가 예상할 수 있는 비밀 번호 리스트를 만든다

ex)

nano pwlist.txt

123456 password 0000 koreakorea miso1111 miso1234 parkmiso 0422miso

이제 Hydra가 SSH계정 비밀번호를 맞출 수 있는지 실험한다.

hydra -l hydratest -P pwlist.txt ssh://127.0.0.1

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ hydra -l hydratest -P pwlist.
txt ssh://127.0.0.1
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in mi
litary or secret service organizations, or for illegal purposes (this is non-bin
ding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-26 22:24:
52
[WARNING] Many SSH configurations limit the number of parallel tasks, it is reco
mmended to reduce the tasks: use -t 4
[DATA] max 9 tasks per 1 server, overall 9 tasks, 9 login tries (l:1/p:9), ~1 tr
y per task
[DATA] attacking ssh://127.0.0.1:22/
[22][ssh] host: 127.0.0.1   login: hydratest   password: miso1234
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-11-26 22:24:
56
```

결과 화면

[DATA] max 9 tasks per 1 server, overall 9 tasks, 9 login tries (l:1/p:9), ~1 try per task

뜻 : 총 시도할 로그인 횟수: 9번 ,사용자 1명(hydratest), 비번 후보 9개

[DATA] attacking ssh://127.0.0.1:22/

뜻 : 공격 대상이 로컬 머신(127.0.0.1)의 SSH(22번 포트)

[22][ssh] host: 127.0.0.1 login: hydratest password: miso1234

SSH 서버: 127.0.0.1, 계정: hydratest ,

비밀번호: miso1234 → Hydra가 이 비밀번호를 wordlist에서 찾아내는 데 성공

이제 Hydra가 찾은 비밀번호가 진짜 맞는지 직접 SSH로 로그인 해본다.

ssh hydratest@127.0.0.1

여기서 password로 miso1234를 입력해서 정상적으로 로그인 되면 hydra가 맞춘 비밀번호가 정확하다는 것이다.

```
newcomputer@newcomputer-VMware-Virtual-Platform:~$ ssh hydratest@127.0.0.1
The authenticity of host '127.0.0.1 (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:CvJ7LNrvyRiKJPzCe++onqX7cQOTAsKmiK/mn381l1U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '127.0.0.1' (ED25519) to the list of known hosts.
hydratest@127.0.0.1's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-36-generic x86_64)
```

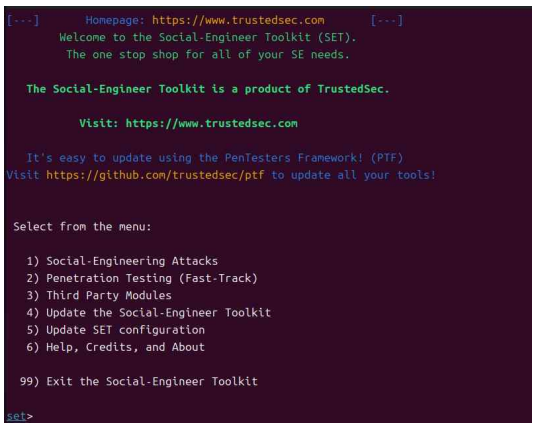
정상적으로 로그인 되었음을 확인할 수 있다.

12주차 SET 설치하기

Social Engineering Toolkit (SET)은 오픈 소스 침투 테스트 도구이다. 이 도구는 소셜 엔지니어링 공격 시나리오를 자동화하고 시뮬레이션하도록 설계되었으며 모의 해킹 목적으로 주로 사용된다. SET은 대표적으로, 실제 웹사이트와 유사한 가짜 로그인 페이지를 제작하여 인증 정보를 탈취하는 피싱 공격 시뮬레이션을 수행할 수 있으며, 악성 코드를 내장한 감염된 미디어 사용 유도나 PowerShell 스크립트 기반 공격 등 다양한 시나리오를 지원한다.
이번 주차에는 Google 피싱 로그인 페이지를 제작하고, 사용자가 로그인 버튼을 누를 경우 로컬 서버의 post.php로 전송되도록 구성하는 실습을 해볼 것이다.

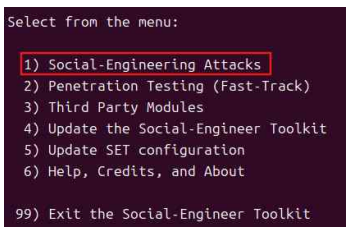
먼저 SET을 설치한다.
git clone https://github.com/trustedsec/social-engineer-toolkit.git
cd social-engineer-toolkit
sudo python3 setup.py install #설치 과정에서 python 라이브러리들이 자동으로 설치된다.

이제 SET을 실행시켜본다
sudo setoolkit

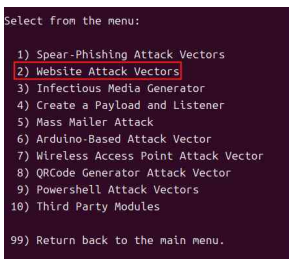


성공하면 다음과 같이 뜬다.

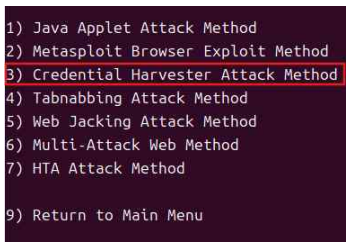
이제 실습용 가짜 피싱 페이지를 만들 것이다 그러므로 다음 옵션들을 차례대로 선택한다.



→



→



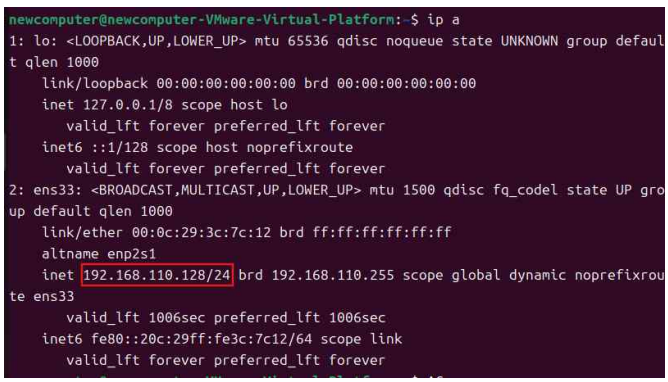
이 옵션들이 의미하는 것은 다음과 같다
- Social-Engineering Attacks → 사람을 속이는 공격 기법 메뉴

- Website Attack Vectors → 웹 기반 피싱/클로닝 메뉴
- Credential Harvester → 로그인 정보(ID/PW) 탈취 기능
- Web Templates → 이미 만들어진 템플릿 사용

여기서 우리는 Google템플릿을 사용할 것이다. Google을 선택하면 SET은 자동으로 구글 로그인 페이지를 복사해서 HTML/CSS/JS 로컬 서버에 배치하고 로그인 정보 제출 시 데이터를 받을 수 있게 준비한다. 이는 해커가 HTML을 직접 만들지 않아도 그럴싸한 가짜 로그인 화면을 만드는 과정과 같다.

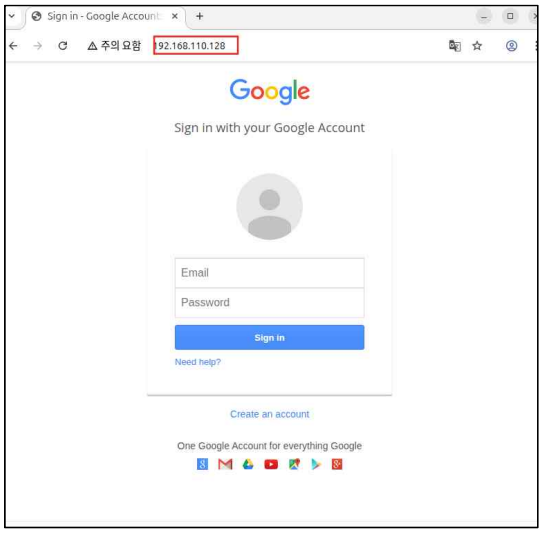
기본적으로 SET이 만드는 credential 파일은 입력된 ID/PW를 공격자 pc에 저장한다.

이제 실습을 검증해보자 . 먼저 현재 내가 사용하고 있는 가상머신의 IP를 확인한다
ip a # ip확인



ip는 192.168.56.101 임을 확인할 수 있다.

이제 브라우저에 접속해보자
http://192.168.56.101



구글 로그인 페이지와 거의 유사한 페이지가 뜨는 것을 확인할 수 있다.
이것과 연결할 가짜 페이지를 만든다
sudo nano /var/www/html/fake.html

이제 sign in 버튼을 누르면 fake.html로 이동시키도록 해야한다. 구글 로그인 페이지에서 Sign in 버튼을 누르면 로컬의 피싱 페이지(post.php)로 이동하도록 실습을 진행하고자 했으나, 실제로는 버튼 클릭 시 항상 구글의 실제 로그인 서버로 이동하는 문제가 발생했다. 이는 구글 로그인 페이지가 단순 HTML 폼이 아니라, 자바스크립트가 로그인 과정 전체를 제어하는 동적 구조로 설계되어 있기 때문이다. 또한 페이지 내부에는 CSRF 토큰, OAuth 인증 값, 보안 쿠키 검증 등 복잡한 보안 절차가 포함되어 있어, SET가 제공하는 단순 클론 방식으로는 이를 우회하거나 재현할 수 없다. 그 결과, 우리가 form action을 post.php로 변경해도 브라우저가 이를 무시하고 구글 서버로만 요청을 보내기 때문에 피싱 흐름을 구성하기가 어려웠다.

13주차 감상문

이번 한 학기 동안 진행한 '리눅스 한 학기 살기' 프로젝트를 통해서 직접 도구를 설치하고 보안 관련 실습을 설계하며 실제 CTF 문제를 만들어 보았다. 리눅스는 윈도우와 달리 사용자가 파일 시스템 구조(/home, /etc, /var/www/html 등)를 직접 이해해야 하고, 서비스의 실행·중지·포트 바인딩을 명령어로 제어해야 한다. 처음에는 이러한 구조가 생소했지만, 매주 새로운 패키지를 설치하며 apt, systemctl, ss, chmod, nano, scp 같은 명령어들이 자연스럽게 익숙해졌다. 특히 서비스 기반 실습(vsftpd, OpenSSH, Bind9 등)을 하며 "리눅스는 서비스가 어떻게 동작하고 어떤 설정 파일에서 제어되는지"를 직접 체험할 수 있었다

"각 주차마다 하나의 프로그램을 직접 설치하고, 그 도구를 활용해 CTF 문제를 설계한다"는 목표로, 이번 프로젝트는 리눅스와 보안을 동시에 익힐 수 있는 흥미로운 경험이었다. Vim에서는 공백 기반 스테가노그래피 문제를 만들었고, curl을 이용해 헤더 변조를 통한 웹 해킹 우회 문제를 제작했다. tar/gzip 실습에서는 깊은 디렉터리 구조 속 숨겨진 증거 파일을 찾아내는 포렌식 시나리오를 구성했으며, vsftpd를 사용해 FTP 로그인 구조와 숨김 파일 노출 위험성을 살펴보았다. Netcat과 Nmap을 활용해 포트 스캔 및 서버 플래그 획득 문제를 제작했고, Bind9에서는 DNS zone 파일 변조 공격을 시뮬레이션하며 "도메인 이름 하나만 바뀌어도 전체 서비스가 공격자 의도대로 이동할 수 있다"는 것을 알 수 있었다. 또한 ffmpeg를 통해 오디오 인코딩,암호화,역재생 문제를 제작했으며, Hydra와 OpenSSH로 실제 비밀번호 무차별 대입 공격을 재현했다. 마지막으로 SET를 활용하여 피싱 페이지를 구성하고 동작 원리를 직접 확인해 볼 수 있었다.

이번 프로젝트를 통해 단순히 보안 도구를 사용하는 수준을 넘어, 리눅스 시스템을 직접 구성하고 통제하면서 보안의 원리에 대해서도 학습할 수 있었다. 무엇보다도 실습을 통해 배운 보안 개념들이 모두 리눅스의 파일 시스템 구조, 프로세스 관리, 권한 모델 위에서 작동한다는 사실 또한 알 수 있었다. 이번 한 학기 리눅스 실습은 보안 지식뿐 아니라 리눅스 시스템에 대해서도 깊이 이해하게 되었다.